



EKF iteration code, load model

- Implementation of EKF on nonlinear model refactors code.
 - “Wrapper” code, coordinates the entire simulation process.
 - Initialization routine (initEKF.m), called once at startup.
 - Update routine (iterEKF.m), called every sample interval.
- Code begins by loading model parameters into local variables.

```
function [xhat,bounds,ekfData] = iterEKF(uk,zk,dT,ekfData)
    model = ekfData.model;

    % Get data stored in ekfData structure
    priorU = ekfData.priorU;
    SigmaX = ekfData.SigmaX;
    SigmaV = ekfData.SigmaV;
    SigmaW = ekfData.SigmaW; % 2x2 matrix computed using "a little trick"
    Sw = ekfData.Sw; % scalar of continuous-time process noise
    xhat = ekfData.xhat;
```



EKF iteration code, steps 1a and 1b

- EKF iteration code continues:
 - Compute $\hat{A}_{k-1}$ and $\hat{B}_{k-1}$ and implement Steps 1a and 1b.

```
% EKF Step 0: Compute Ahat(k-1), Bhat(k-1)
Ahat = [1 dT; -model.k1*dT/model.m-3*model.k2*dT*xhat(1)^2/model.m ...
        1-model.b*dT/model.m];
Bhat = [0; dT/model.m];

% Step 1a: State prediction time update
xhat = [1 dT; -model.k1*dT/model.m 1-model.b*dT/model.m]*xhat + ...
        [0; -model.k2*dT*xhat(1)^3/model.m] + ...
        [0; dT/model.m]*priorU;

% Step 1b: Prediction-error covariance time update
%      sigmaminus(k) = Ahat(k-1)*sigmaplus(k-1)*Ahat(k-1)' + ...
%                  Bhat(k-1)*sigmawtilde*Bhat(k-1)'
SigmaX = Ahat*SigmaX*Ahat' + Bhat*(Sw/dT)*Bhat'; % default method
% SigmaX = Ahat*SigmaX*Ahat' + SigmaW; % alternate method
```



EKF iteration code, steps 1c–2c

- EKF iteration code continues
 - Compute $\hat{C}_k$ and $\hat{D}_k$ and implement Steps 1c–2c.

```
% Step 1c: Output estimate
zhat = atan(xhat(1)/model.d);

% Step 2a: Estimator gain matrix
Chat = [model.d/(model.d^2 + xhat(1)^2) 0]; Dhat = 1;
SigmaZ = Chat*SigmaX*Chat' + Dhat*SigmaV*Dhat';
L = SigmaX*Chat'/SigmaZ;

% Step 2b: State estimate measurement update
r = zk - zhat; % innovation: use to check for sensor errors...
if r^2 > 100*SigmaZ, L(:)=0.0; end
xhat = xhat + L*r;

% Step 2c: Estimation-error covariance measurement update
SigmaX = SigmaX - L*SigmaZ*L';
```



EKF iteration code, cleanup

- EKF iteration code continues
 - Add some robustness checking to estimate.
 - Store results for next time through loop.

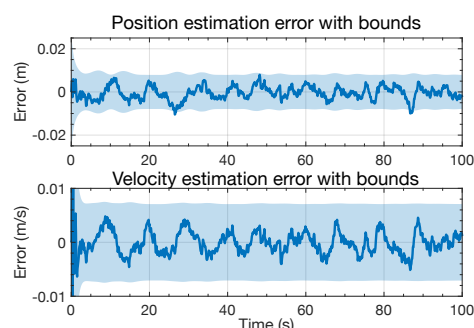
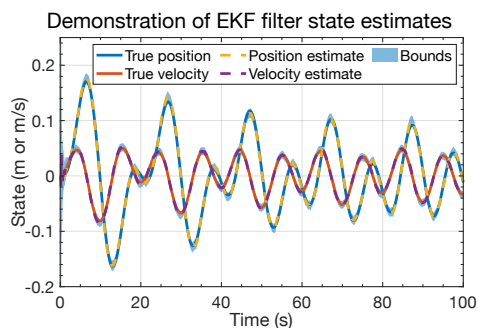
```
% Q-bump code
if r^2 > 16*SigmaZ % bad output estimate by 4 std. devs, bump Q
    fprintf('Bumping SigmaX\n');
    SigmaX = SigmaX*ekfData.Qbump;
end
[~,S,V] = svd(SigmaX); % Implement Higham method to help robustness
HH = V*S*V';
SigmaX = (SigmaX + SigmaX' + HH + HH')/4;

% Save data in ekfData structure for next time...
ekfData.priorU = uk;
ekfData.SigmaX = SigmaX;
ekfData.xhat = xhat;
bounds = 3*sqrt(diag(SigmaX));
end
```



Example EKF on nonlinear model results

- EKF was executed for the dataset presented in Lesson 3.1.5.
 - RMS state-estimation error = 0.0028.
 - Percent of time error outside bounds = 0.45%.



Summary

- Implementation of EKF on nonlinear model refactors code.
 - Initialization routine (`initEKF.m`), called once at startup.
 - Update routine (`iterEKF.m`), called every sample interval.
 - “Wrapper” code, coordinates the entire simulation process.
- You have now seen the details of the entire codeset plus some example results.
- EKF works quite well as state estimator using this nonlinear model.